

Session 3 · Deck 3.B4

The Data Workflow: Acquire, Clean, Persist, Export

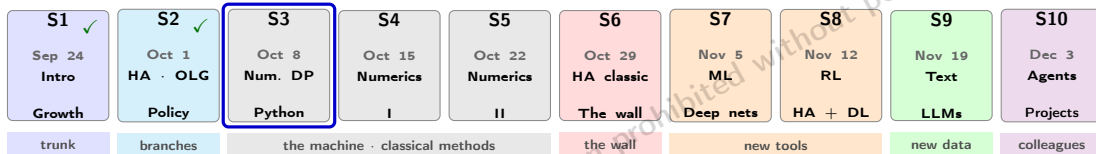
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.B4 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 3.B1 your toolkit · 3.B2 from loops to arrays · 3.B3 PyTorch, and coding with AI · 3.B4 the data workflow · 3.B5 hands-on: build it yourself.

Hands-on, 3.B5:

1 Version 0: grid VFI
against the closed form

2 Data → parameters:
 $g, n, \alpha, \delta, \beta$

3 The calibrated economy,
solved and checked

4 Test the model: untar-
geted ratio, Euler residual

Your Model Produces Moments. So Does the Data.

Everything this morning ended in a number the *model* produced. Calibration and estimation are the comparison of those numbers with the ones the **data** produces — and that comparison is not free. It needs the series pulled, the units fixed, the trend removed, and the result saved somewhere you can find it again.

	Long run (today)	Cycles (Session 4)
Model	steady-state ratios	simulate, HP-filter, σ , ρ
Data	averages of the same ratios	download, clean, HP-filter, σ , ρ

The two halves must match

The same filter, the same frequency, the same detrending. A business-cycle statistic compared against one computed a different way is not a comparison at all.

Homework 1 asks for both halves: Part A (A3) pulls the U.S. series from FRED and HP-filters them; Part B simulates the model and filters its series the same way. This deck is the data half; the hands-on (3.B5) uses its **long-run** part.

The Data Workflow

The pipeline in an empirical macro project:

1. **Acquire** the data — FRED, the World Bank, a CSV from a coauthor.
2. **Clean** — missing values, types, dates.
3. **Persist** — save the clean version, so you never redo steps 1–2.
4. **Analyze and visualize** — moments, regressions, plots.
5. **Export** — figures and tables for the paper.

Step 3 is the one people skip

Re-downloading on every run makes your results depend on a web service and on the day you ran it. Save the clean data once; commit the *script* that produced it.

Vintages change. FRED revises GDP. A figure you cannot reproduce next March is a figure you cannot defend in a referee report.

Acquire: FRED GDP Data

```
import pandas as pd

URL = ("https://fred.stlouisfed.org/graph/"
      "fredgraph.csv?id={}")

def fred(sid):    # one series, by its ID
    return pd.read_csv(URL.format(sid),
                        index_col=0,
                        parse_dates=True)[sid]

# 'GDPC1' is real GDP (quarterly)
gdp = fred('GDPC1').loc['1990': '2025']
gdp = gdp.to_frame()    # a DataFrame
print(gdp.head())
```

- FRED serves every series as a CSV file at a fixed address, and pandas reads it directly: no extra package, no API key.
- FRED identifies every series by a **series ID** — GDP, GDPC1 (real), UNRATE, FEDFUNDS.
- What comes back is a **pandas DataFrame**: a table with a date index.
- pandas_datareader does the same in one call (`web.DataReader('GDPC1', 'fred', start, end)`) but is one more package to install; `fredapi` is the one that asks for a key.

GDP, without the suffix, is *nominal*. For business cycles you almost always want GDPC1, which is why we fetched it.

Clean: Growth Rates and Missing Values

```
import pandas as pd
import numpy as np

# quarterly growth rate
gdp['growth'] = gdp['GDPC1'].pct_change()

# the first difference is NaN
clean = gdp.dropna()

# a subsample
post_2000 = clean[clean.index >= '2000-01-01']

print(post_2000[['GDPC1', 'growth']].head())
```

A DataFrame column behaves like a NumPy array with a label and a date index attached, so `pct_change` and `dropna` are vectorised for the same reason the Bellman update was.

`dropna()` is silent. **Print how many rows it removed.** A filter that quietly deletes a third of your sample is the most expensive kind of bug, because the code still runs and the plot still looks fine.

The HP Filter: What It Is and What It Does

What it is. Given a series y_t (in logs), Hodrick and Prescott choose the trend τ_t that solves

$$\min_{\{\tau_t\}} \sum_t (y_t - \tau_t)^2 + \lambda \sum_t [(\tau_{t+1} - \tau_t) - (\tau_t - \tau_{t-1})]^2.$$

The first sum rewards **fit**; the second penalises changes in the trend's *growth rate*. The cycle is what is left:

$$c_t = y_t - \tau_t.$$

What it does. $\lambda = 0$ gives $\tau = y$ and no cycle; $\lambda \rightarrow \infty$ gives a straight line. In between it is a linear, two-sided filter that hands **slow swings to the trend** and keeps **faster fluctuations in the cycle**. The cutoff is the period at which the cycle keeps half of a fluctuation:

$$4\lambda(1 - \cos \omega)^2 = 1.$$

Why $\lambda = 1600$ for quarterly data

$\lambda = (\sigma_c / \sigma_{\Delta^2 \tau})^2$. Hodrick and Prescott judged a 5% cycle as large as a $\frac{1}{8}$ -point change in quarterly trend growth: $\lambda = (5 / \frac{1}{8})^2 = 1600$.

data	λ	cutoff
quarterly	1,600	9.9 years
annual (Ravn–Uhlig)	6.25	9.8 years
monthly (Ravn–Uhlig)	129,600	9.9 years
annual (Backus–Kehoe)	100	19.8 years

Why rescale: the filter depends on $\lambda \omega^4$, so λ moves with the *fourth* power of the sampling rate:

$$1600 / 4^4 = 6.25.$$

The HP Filter: Trend and Cycle

```
import numpy as np
from statsmodels.tsa.filters.hp_filter \
    import hpfilter

gdp_log = np.log(clean['GDPC1'])

# lambda = 1600 for quarterly data
cycle, trend = hpfilter(gdp_log, lamb=1600)

clean['gdp_trend'] = trend
clean['gdp_cycle'] = cycle

# the business-cycle moments
print(clean['gdp_cycle'].std())
print(clean['gdp_cycle'].autocorr(1))
```

HW1 (Part A, A3; Part B) needs the moment lines and the caveat.

The code: `hpfilter` returns the cycle first, then the trend; λ is the λ of the previous frame, 1600 for quarterly GDP.

The last two lines are the object of comparison: the standard deviation and the first-order autocorrelation of the cyclical component. Your model must produce the same two numbers, computed the same way.

It is a convention, not a theory — it invents cycles at the ends of the sample and its filtered series can mislead (Hamilton 2018). Report what you used.

Persist: Save the Clean Version

```
# CSV: shareable, readable by anything
post_2000.to_csv("gdp_post2000.csv")

# pickle: Python-only, preserves dtypes
post_2000.to_pickle("gdp_post2000.pkl")

# later, in a different script
df = pd.read_csv("gdp_post2000.csv",
                 index_col=0,
                 parse_dates=True)
```

- **CSV** for anything a coauthor, a referee or your future self has to open.
- **Pickle** for speed and exact types, inside one project only — it is not a durable format.
- `parse_dates=True` is what turns the index back into dates rather than strings.

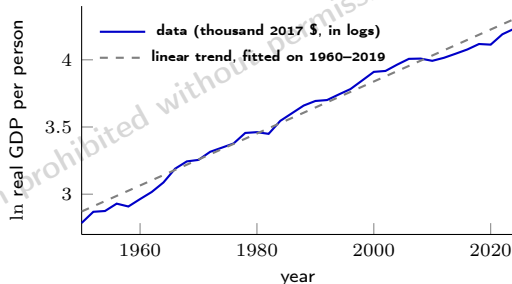
This is the same discipline as 3.A1's number ledger: the computation writes a file, and everything downstream reads that file instead of recomputing.

Analyze: Growth Is a Straight Line in Logs

```
import numpy as np

pop = fred('B23ORCOA052NBEA') # thousands
y = fred('GDPCA') / pop        # per person
y.index = y.index.year         # annual
S = slice(1960, 2019)
yrs = np.arange(1960, 2020)
b = np.polyfit(yrs, np.log(y.loc[S]), 1)
g = np.exp(b[0]) - 1            # trend growth
gap = np.log(y.loc[S]) - np.polyval(b, yrs)
```

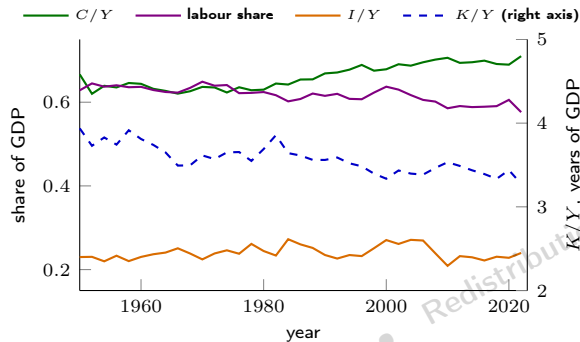
Detrending for the long run is one least-squares line through $\ln y_t$. Its slope is the growth rate of the balanced growth path: $g = 1.95\%$ a year over 1960–2019; the same line through population gives $n = 1.04\%$.



The line gives the **long run**. What is left over — the gap — holds the long swings and the cycles; splitting those is the HP filter's job, next week.

FRED: GDPCA (BEA, real GDP), B23ORCOA052NBEA (BEA, population). Numbers from tools/figures/h03_numbers.json.

Analyze: What Does Not Grow



Output per person rose more than fourfold since 1950. These four ratios did **not** trend.

The great ratios

On a **balanced growth path** every level grows and every ratio is constant. So a steady state cannot match levels — it matches **ratios**: capital to output, investment to capital, the labour share. Average them over the sample, and you have the targets.

Take K , I and Y from **one source**, so they cover the same economy. Here: the Penn World Table, through FRED.

Penn World Table 11.0 (Feenstra, Inklaar and Timmer 2015), via FRED; the series IDs are in 3.B5. C/Y is household consumption. Every second year plotted.

Analyze: From Ratios to Parameters

Along the balanced growth path of the growth model (3.B5 writes it out), two equations tie the ratios to the parameters:

- **Capital accumulation.** Investment replaces what wears out *and* equips the new effective workers:
$$I/K = (1 + g)(1 + n) - 1 + \delta.$$
- **Euler equation.** Consumption per person grows at g , so
$$(1 + g)^\gamma = \beta (1 + \alpha Y/K - \delta).$$

Parameter	Target, 1960–2019	Value
g, n	trend growth: y , population	1.95%, 1.04%
α	1 – labour share	0.384
δ	$I/K = 0.0685$, accumulation	0.038
γ	set, not calibrated	2
β	$K/Y = 3.54$, Euler equation	0.971

Numbers from `tools/figures/h03_numbers.json`; data as downloaded 2026-10-04.

Exactly identified

Five targets, five parameters: each equation is solved for one unknown, in order. The implied net return on capital is $\alpha Y/K - \delta = 7.0\%$.

So the targeted ratios come back **by construction**. That tests your code, not the model. Test the model on a ratio you did *not* target — in the hands-on, C/Y .

Visualize: GDP Growth

```
import matplotlib.pyplot as plt

x = df.index
y = df['growth'] * 100          # percent

plt.figure(figsize=(10, 5))
plt.plot(x, y, label="quarterly growth (%)")
plt.axhline(0, linestyle='--', color='grey')
plt.title("US Real GDP Growth")
plt.xlabel("year")
plt.ylabel("growth (%)")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
```

Nothing here is specific to economics — it is the same matplotlib you used to plot a policy function this morning, pointed at a date index.

Label the axes and state the units on every figure you make, including the throwaway ones. The throwaway figure is the one that ends up in a seminar.

Export: Figures for the Paper

```
# raster, for slides  
plt.savefig("gdp_growth.png", dpi=300,  
            bbox_inches='tight')
```

```
# vector, for LaTeX  
plt.savefig("gdp_growth.pdf",  
            bbox_inches='tight')
```

```
plt.show()
```

savefig must come **before** show() — showing a figure clears it, and you get an empty file.

Vector for a paper, always

A PDF stays sharp at any zoom and its text is real text. A 300-dpi PNG is for slides and for anything a journal insists on rasterising.

Every number and every plotted curve in Session 3's decks was produced this way — a script wrote the numbers to a file, and the figure was drawn from that file, never by hand.

Takeaways

- Calibration is a **comparison**, and a comparison is only valid if both sides were computed the same way — same filter, same frequency, same detrending.
- The pipeline is **acquire** → **clean** → **persist** → **analyze** → **export**, and step 3 is the one that makes the other four reproducible.
- `dropna()` is silent. Print what it removed, every time.
- **The long run:** one straight line through log output per person gives g ; the great ratios do not trend, so their averages are the targets; the steady state, inverted, gives α , δ , β . Then check a ratio you did not target.
- **The cycle:** the HP filter with $\lambda = 1600$ (quarterly; 6.25 annual), and the two numbers that come out of it — the standard deviation and the autocorrelation of the cycle — are what your model has to match.
- It is a convention, not a theory. Say what you used, and save the file that produced it.

Bridge: Now Build It

Block A gave you a solver and a closed form to grade it against. This deck gave you the numbers that turn the solver's model into the U.S. economy. The hands-on puts the two together.

- **Step 1** is 3.A2's algorithm, written with 3.B2's broadcasting.
- **Step 2** is this deck's last three *Analyze* frames.
- **Step 3** is 3.A4's model with CRRA utility and growth.
- **Step 4** tests it on a ratio this deck did *not* target, and on 3.A3's Euler residual.

Next (3.B5): Hands-on

Open your environment from 3.B1, a browser, and an AI assistant if you use one. Work in pairs. Every step ends in a number you can check before you move on.